

Infrastructure

Infrastructure, middleware, and platforms

Infrastructure

The term *infrastructure* is a broad, *catch-all* term that includes all the underlying systems, hardware, and foundational software that sit *beneath an application*.

Like the supporting systems of a city:

- plumbing,
- the electrical grid,
- and waste management,

infrastructure provides the *compute power*, storage, and networking needed to run software.

By **decoupling** applications from these low-level resources, infrastructure allows developers to focus on building features without constantly worrying about the physical machines running their code.

Hardware

Hardware, Software, and Virtualization

Historically, infrastructure referred mainly to **physical hardware** (*bare metal*), including servers, Ethernet cables, and hardware firewalls housed in data centers.

This approach was:

- expensive,
- slow to scale,
- and vulnerable to single points of failure.

The industry shifted with the introduction of **virtualization**. Using **hypervisors**, a single physical server could be divided into multiple isolated Virtual Machines (VMs).

This eventually evolved into **software-defined infrastructure**, where networking, storage, and security resources can be created and managed through software rather than physical hardware.

Hybrid Cloud

While cloud providers such as AWS, Azure, and Google Cloud offer massive scalability, many organizations use a **hybrid cloud** strategy.

A hybrid cloud combines:

- on-premises infrastructure or private clouds for legacy systems, compliance requirements, data sovereignty, or intellectual property protection,
- with public cloud resources for scalable and customer-facing workloads.

The biggest challenge is integration.

Data must move securely and consistently between on-premises and cloud environments, requiring:

- strong networking,
- unified security controls,
- and reliable synchronization mechanisms.

Example

A modern web application infrastructure is built from several foundational layers.

Together, these components provide the foundation needed to deploy and run applications.

Compute

- Physical servers or cloud virtual machines such as AWS EC2.

Storage

- Storage Area Networks (SANs) for databases.
- Cloud object storage such as Amazon S3 for files and static assets.

Networking

- Virtual Private Clouds (VPCs),
- subnets,
- load balancers,
- and firewalls.

Middleware

Middleware

If infrastructure is the foundation, middleware is the *glue* that connects applications, databases, and services together.

Middleware acts as an intermediary layer that helps different systems communicate, even when they use different operating systems, protocols, or data formats.

The term is *intentionally broad* and can mean different things depending on the context, ranging from enterprise integration platforms to web request handlers.

In General

In enterprise architecture, middleware usually refers to large software systems designed to *connect and decouple applications*.

A common example is a **message broker** such as:

- Apache Kafka,
- RabbitMQ,
- IBM MQ.

These systems allow independent services to communicate through message queues instead of direct HTTP connections.

Middleware acts like a centralized traffic controller, allowing old and new systems to exchange data without needing to understand each other's internal design.

In Web Applications

In frameworks such as Express.js, Django, or Laravel, middleware refers to code that runs *between receiving an HTTP request* and sending a response.

Middleware handles cross-cutting concerns that should not clutter business logic.

Common responsibilities include:

- authentication and authorization,
- logging and monitoring,
- performance tracking,
- CORS configuration,
- rate limiting,
- and security checks.

This keeps application code cleaner and easier to maintain.

In Object-Relational Mapping (ORM)

ORM libraries such as Hibernate, Prisma, and SQLAlchemy act as middleware *between application code and relational databases*.

Applications organize data as objects, while databases store data in tables.

This difference is known as the **Object-Relational Impedance Mismatch**.

An ORM solves this problem by translating object operations into SQL queries, allowing developers to work with objects while the ORM handles database communication behind the scenes.

In Game Engines

The meaning of middleware changes slightly in game development.

From a technical perspective, game engines such as Unreal Engine or Unity *act as middleware* because they sit between game code and the operating system or graphics APIs.

However, game developers usually refer to these as engines or platforms.

Instead, the term middleware is often reserved for specialized third-party tools such as:

- Havok or PhysX for physics simulation,
- Wwise or FMOD for audio systems,
- SpeedTree for vegetation rendering.

These tools provide advanced functionality without requiring developers to build everything from scratch.

Platforms

Platforms

A platform is the environment *where software actually runs*.

It sits on top of infrastructure and serves as the deployment target before application-specific logic is added.

The main purpose of a platform is **abstraction and standardization**.

Platforms automate tasks such as:

- server provisioning,
- operating system patching,
- runtime management,
- and environment configuration.

This ensures applications behave consistently across development, testing, and production environments.

Types of Platforms

Modern enterprise integration commonly relies on three platform categories.

Platform as a Service (PaaS)

Examples:

- AWS Elastic Beanstalk
- Heroku

Developers deploy code while the platform handles:

- scaling,
- load balancing,
- operating system maintenance,
- and infrastructure management.

Integration Platform as a Service (iPaaS)

Examples:

- MuleSoft
- Dell Boomi

These platforms specialize in connecting enterprise applications through pre-built connectors and integration tools.

Container and Orchestration Platforms

Where developers can have *on-premise infrastructure* but get the benefits of PaaS and iPaaS management

Examples:

- Kubernetes
- Red Hat OpenShift

These platforms manage:

- deployment,
- scaling,
- networking,
- and orchestration of containerized applications across clusters.

Platform vs Infrastructure vs Middleware

A useful analogy is a building.

Infrastructure

- The land, foundation, utility pipes, and power lines.

Platform

- The building structure with elevators, electrical systems, and standard facilities that make it usable.

Middleware

- The internal systems such as intercoms, security checkpoints, and smart communication systems that allow different parts of the building to interact.

Together, these layers create an environment where applications can operate smoothly.

Integration Techniques

Wrappers

The wrapper technique places a modern interface *around an existing system*.

This is commonly used when organizations *depend on legacy software* that still performs critical tasks but **cannot** communicate using modern standards such as REST APIs, GraphQL, or JSON.

A wrapper:

1. Accepts modern requests,
2. Translates them into the format expected by the legacy system,
3. Executes the operation,
4. Converts the result back into a modern format.

This allows older systems to *integrate* with modern applications without being completely replaced.

Glue Code

Glue code is software written specifically to connect systems that were *never designed to work together*.

Its purpose is not to perform business logic such as:

- processing orders,
- calculating taxes,
- or authenticating users.

Instead, it:

- receives data from one system,
- converts formats or data types,
- and passes the result to another system.

While useful for quick integrations and prototypes, excessive glue code can *create technical debt* and lead to fragile, difficult-to-maintain systems.

Facades

A facade provides a *simplified and unified interface* to a complex subsystem.

For example, processing an online order may require interactions with:

- inventory systems,
- payment gateways,
- shipping providers,
- and CRM platforms.

Without a facade, the client application would need to *manage all of these integrations directly*.

With a facade, the client communicates with a single service, such as an **OrderProcessingService**.

The facade coordinates all required operations behind the scenes and returns a *clean, consolidated response*, reducing complexity and improving maintainability.